



**Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)**

КУРСОВОЙ ПРОЕКТ

«Создание базы данных для системы бронирования»

Выполнил:

студент 3-го курса специальности
09.02.07 «Информационные системы и
программирование»

Гаев Константин Романович

подпись: _____

Проверил:

преподаватель дисциплины,
преподаватель ЧУПО «ВШП»,
к.ф.н. Ткачев П.С.

оценка: _____

подпись: _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ.....	4
1.1. Анализ предметной области «Система бронирования отелей».....	4
1.2. Реляционная модель данных и нормализация.....	4
1.3. Обеспечение безопасности и ролевая модель в СУБД.....	5
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ БД ДЛЯ СИСТЕМЫ БРОНИРОВАНИЯ.....	6
2.1. Концептуальное проектирование и ER-диаграмма.....	6
2.2. Логическая структура и приведение к 3НФ.....	7
2.3. Физическая реализация и объекты БД.....	8
2.3.1. Представление (View).....	8
2.3.2. Пользовательская функция (Function).....	9
2.3.3. Хранимая процедура (Procedure).....	10
2.3.4. Триггер (Trigger).....	11
2.3.5. Транзакция (Transaction).....	11
2.4. Типовые запросы и бизнес-логика.....	12
2.5. Ролевая модель доступа.....	13
ЗАКЛЮЧЕНИЕ.....	14
СПИСОК ИСТОЧНИКОВ.....	15
Приложение 1. Ссылка на репозиторий.....	16

ВВЕДЕНИЕ

Актуальность темы. Динамичный рост индустрии гостеприимства и цифровая трансформация туристического сектора обуславливают острую необходимость в автоматизации процессов бронирования. Внедрение специализированных информационных систем позволяет максимизировать загрузку номерного фонда, исключить риски, связанные с «человеческим фактором», гарантировать прозрачность финансовых операций и существенно повысить качество обслуживания гостей. Фундаментом подобной системы выступает надежная база данных, которая гарантирует целостность, конфиденциальность и быстрое действие при обработке транзакций.

Объект исследования — процессы проектирования, разработки и обеспечения безопасности баз данных в информационных системах бронирования.

Предмет исследования — методологии и инструменты концептуального и физического проектирования реляционных баз данных в сфере гостиничного бизнеса на платформе MySQL.

Цель работы — разработка архитектуры и программная реализация реляционной базы данных для системы управления бронированием отелей, отвечающей строгим критериям нормализации, информационной безопасности и функциональной полноты.

ГЛАВА 1. ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ

1.1. Анализ предметной области «Система бронирования отелей»

Предметная область гостиничного бизнеса включает в себя множество взаимосвязанных бизнес-процессов. Основными сущностями и процессами являются:

- Номерной фонд: Комнаты различных категорий (стандарт, люкс, семейный), каждая из которых имеет свою стоимость, вместимость и текущий статус (свободен, занят, на уборке).
- Гости: Клиенты, оставляющие заявки. Для каждого гостя необходимо хранить персональные данные и контактную информацию.
- Бронирования: Запросы на проживание, содержащие даты заезда и выезда, информацию о госте и номере, а также статус бронирования (ожидает, подтверждено, отменено, завершено).
- Платежи: Финансовые транзакции, связанные с бронированием (предоплата, полная оплата, возврат).

Для корректной работы системы критически важно обеспечить целостность данных. Например, один и тот же номер не может быть забронирован двумя разными гостями на пересекающиеся даты. Кроме того, статус бронирования должен логически соответствовать статусу платежа. Все эти бизнес-правила требуют тщательного проектирования на уровне СУБД.

1.2. Реляционная модель данных и нормализация

Реляционная база данных представляет собой набор двумерных таблиц. Главной проблемой при проектировании является избежание аномалий при вставке, обновлении и удалении данных. Для решения этой проблемы применяется процесс нормализации.

В данном проекте база данных приводится к Третьей нормальной форме (3НФ).

Отношение находится в 3НФ, если оно находится в 2НФ и все неключевые атрибуты неприводимо зависят от каждого потенциального ключа (отсутствует транзитивная зависимость).

Пример устранения транзитивной зависимости:

Если бы мы хранили в таблице Бронирования данные о Номере (тип номера, цена за сутки), то при изменении цены на номер нам пришлось бы обновлять данные во всех будущих и прошлых бронированиях. Поэтому сущность «Номера» и «Типы номеров» выносятся в отдельные таблицы, а в таблице «Бронирования» остаются только внешние ключи.

1.3. Обеспечение безопасности и ролевая модель в СУБД

Безопасность базы данных обеспечивается за счет разграничения прав доступа. В СУБД MySQL создаются отдельные учетные записи, выполняющие роли:

1. `hotel_admin` — обладает полными правами для администратора системы (управление структурой БД, настройка справочников).
2. `hotel_receptionist` — права на чтение справочников и управление бронированиями (создание, изменение статусов), но без права удаления исторических данных или изменения структуры таблиц.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ БД ДЛЯ СИСТЕМЫ БРОНИРОВАНИЯ

2.1. Концептуальное проектирование и ER-диаграмма

На основе анализа предметной области выделено 5 основных сущностей:

1. Room_Types (Типы номеров)
2. Rooms (Номера)
3. Guests (Гости)
4. Bookings (Бронирования)
5. Payments (Платежи)

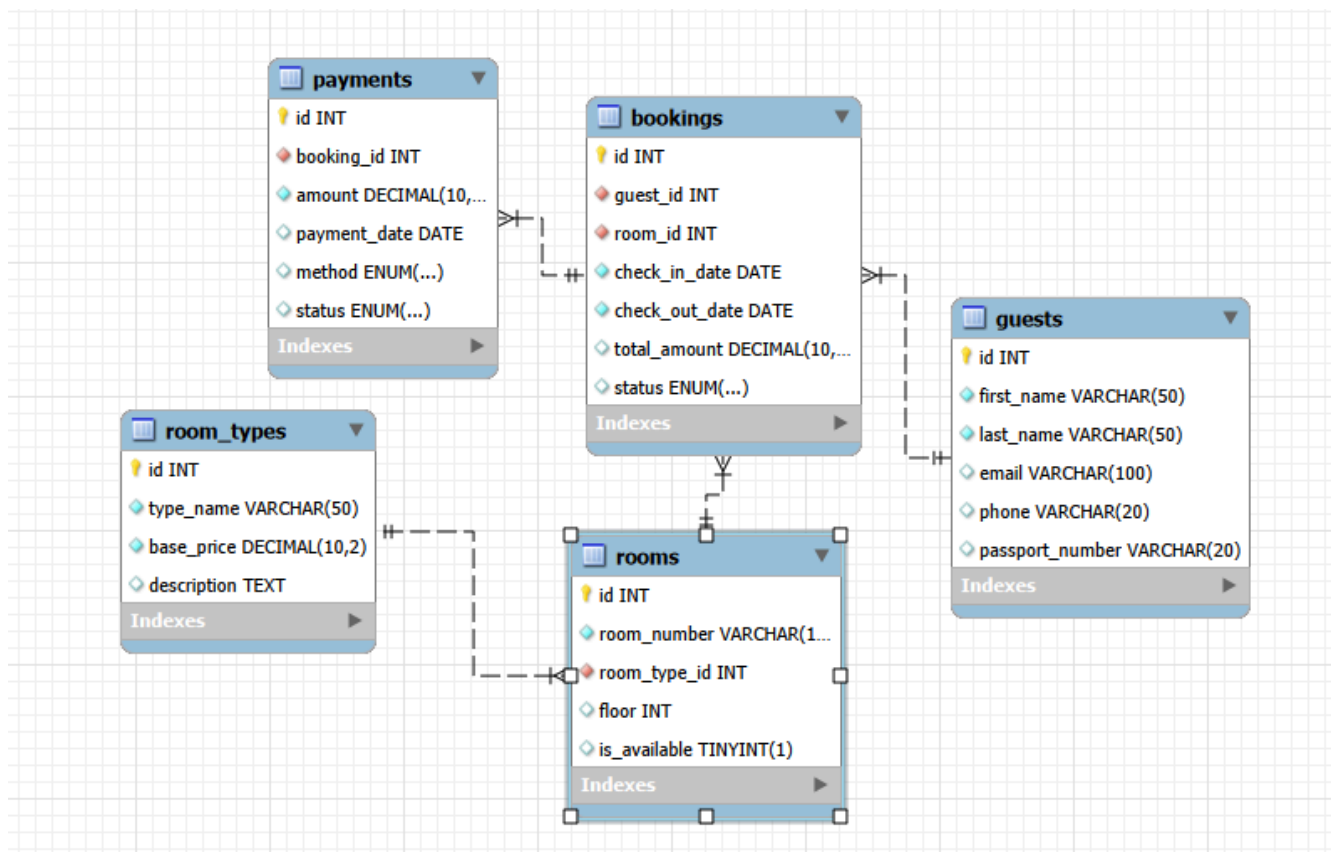


Рисунок 1 -ER-диаграмма

- Один тип номера может включать множество конкретных номеров (1-M).
- Один гость может иметь множество бронирований (1-M).
- Один номер может быть забронирован множество раз в разные даты (1-M).

- Одно бронирование может включать несколько транзакций оплаты (например, предоплата и доплата) (1-M).

2.2. Логическая структура и приведение к 3НФ

Проектирование логической структуры базы данных является критически важным этапом. Исходные данные предметной области содержали дублирование. Для устранения аномалий был проведен процесс нормализации.

Приведение к 1НФ и 2НФ. Все атрибуты приведены к атомарному виду. Введены суррогатные первичные ключи (id типа INT AUTO_INCREMENT). Поскольку ключи простые, отношение автоматически удовлетворяет 2НФ.

Приведение к 3НФ. Устранены транзитивные зависимости. Сущность «Тип номера» (с его базовой ценой) вынесена в отдельную таблицу Room_Types, чтобы изменение цены не затрагивало исторические записи о бронированиях.

Ниже приведено детальное описание структуры каждой таблицы с обоснованием выбора типов данных и ограничений целостности.

Таблица Room_Types (Типы номеров)

Справочник категорий номеров (Стандарт, Люкс). Атрибут base_price имеет тип DECIMAL(10,2) для точного хранения валютных значений. Ограничение UNIQUE на type_name предотвращает создание дублей категорий.

Таблица Rooms (Номера)

Хранит информацию о конкретных комнатах. Атрибут room_number имеет тип VARCHAR(10), так как номера могут быть не только числовыми (например, "105-A"). Внешний ключ room_type_id ссылается на Room_Types. Атрибут is_available имеет тип BOOLEAN (TINYINT(1)) для быстрого фильтрации свободных номеров.

Таблица Guests (Гости)

Содержит персональные данные клиентов. Атрибут passport_number имеет ограничение UNIQUE, так как один паспорт не может быть использован двумя разными гостями, что является важным бизнес-правилом для предотвращения мошенничества.

Таблица Bookings (Бронирования)

Центральная связующая таблица. Атрибуты `check_in_date` и `check_out_date` имеют тип `DATE`. Внешние ключи связывают бронирование с гостем и номером. Атрибут `status` использует тип `ENUM('Pending', 'Confirmed', 'Checked-In', 'Completed', 'Cancelled')`, что жестко ограничивает возможные статусы на уровне СУБД и исключает ввод некорректных значений.

Таблица Payments (Платежи)

Хранит финансовые транзакции. Атрибут `payment_method` использует `ENUM('Cash', 'Card', 'Online')`. Внешний ключ `booking_id` использует правило `ON DELETE CASCADE`, так как при удалении бронирования (например, по ошибке) связанные с ним платежи должны быть удалены автоматически для сохранения финансовой целостности.

Все спроектированные таблицы находятся в 3НФ, что исключает избыточность данных.

2.3. Физическая реализация и объекты БД

Физическая реализация выполнена в СУБД MySQL 8.0. Все таблицы созданы с использованием движка InnoDB (поддержка транзакций и внешних ключей) и кодировки `utf8mb4`.

2.3.1. Представление (View)

Назначение: Упрощение формирования сводной таблицы текущих и будущих бронирований для стойки регистрации. Представление скрывает сложность многотабличных соединений от фронтенд-приложения.

Принцип работы: `View_ActiveBookings` объединяет таблицы бронирований, гостей и номеров. Фильтрация осуществляется на уровне представления: выводятся только те записи, где дата выезда больше или равна текущей дате, а статус не "Отменен".

```
CREATE OR REPLACE VIEW View_ActiveBookings AS
SELECT
    b.id AS booking_id,
```



```

    CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
    r.room_number,
    rt.type_name,
    b.check_in_date,
    b.check_out_date,
    b.status
FROM Bookings b
JOIN Guests g ON b.guest_id = g.id
JOIN Rooms r ON b.room_id = r.id
JOIN Room_Types rt ON r.room_type_id = rt.id
WHERE b.check_out_date >= CURDATE()
AND b.status != 'Cancelled';

```

2.3.2. Пользовательская функция (Function)

Назначение: Автоматический расчет итоговой стоимости бронирования на основе дат и категории номера.

Принцип работы: Функция Func_CalculateBookingPrice принимает даты и ID номера. Внутри объявлена локальная переменная v_nights для хранения количества ночей. С помощью TIMESTAMPDIFF вычисляется разница в днях, которая умножается на базовую цену типа номера.

```

DELIMITER $$
CREATE FUNCTION Func_CalculateBookingPrice(
    p_check_in DATE, p_check_out DATE, p_room_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_nights INT;
    DECLARE v_price_per_night DECIMAL(10,2);

    SET v_nights = TIMESTAMPDIFF(DAY, p_check_in, p_check_out);

    SELECT rt.base_price INTO v_price_per_night
    FROM Rooms r
    JOIN Room_Types rt ON r.room_type_id = rt.id
    WHERE r.id = p_room_id;

    RETURN v_nights * v_price_per_night;
END$$
DELIMITER ;

```

2.3.3. Хранимая процедура (Procedure)

Назначение: Регистрация заезда гостя (Check-In). Процедура проверяет, оплачено ли бронирование, и меняет его статус.

Принцип работы: Процедура Proc_CheckInGuest принимает ID бронирования. Объявлены три локальные переменные для хранения статуса, суммы к оплате и факта оплаты. С помощью условия IF проверяется, совпадает ли сумма бронирования с суммой оплаченных платежей. Если нет, процедура прерывается. Реализован обработчик исключений для отката транзакции в случае сбоя.

```
DELIMITER $$
CREATE PROCEDURE Proc_CheckInGuest(IN p_booking_id INT)
BEGIN
    DECLARE v_booking_status VARCHAR(20);
    DECLARE v_total_amount DECIMAL(10,2);
    DECLARE v_paid_amount DECIMAL(10,2) DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Ошибка при регистрации заезда.' AS Error_Message;
    END;

    START TRANSACTION;

    SELECT total_amount, status INTO v_total_amount,
v_booking_status
    FROM Bookings WHERE id = p_booking_id;

    SELECT COALESCE(SUM(amount), 0) INTO v_paid_amount
    FROM Payments
    WHERE booking_id = p_booking_id AND status = 'Completed';

    IF v_paid_amount < v_total_amount THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Ошибка: Бронирование не оплачено
полностью!';
    END IF;

    IF v_booking_status = 'Cancelled' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Ошибка: Бронирование отменено!';
    END IF;

    UPDATE Bookings SET status = 'Checked-In' WHERE id =
p_booking_id;

    COMMIT;
```

```
END$$  
DELIMITER ;
```

2.3.4. Триггер (Trigger)

Назначение: Защита от создания бронирований с некорректными датами (дата выезда раньше или равна дате заезда).

Принцип работы: Триггер `trg_before_booking_insert` срабатывает перед вставкой в `Bookings`. Если `check_out_date <= check_in_date`, генерируется ошибка `SIGNAL SQLSTATE`, прерывающая операцию.

```
DELIMITER $$  
CREATE TRIGGER trg_before_booking_insert  
BEFORE INSERT ON Bookings  
FOR EACH ROW  
BEGIN  
    IF NEW.check_out_date <= NEW.check_in_date THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Ошибка: Дата выезда должна быть позже  
даты заезда!';  
    END IF;  
END$$  
DELIMITER ;
```

2.3.5. Транзакция (Transaction)

Назначение: Обеспечение целостности при обработке платежа. Платеж не должен быть сохранен, если не удалось обновить статус бронирования.

Принцип работы: Явное управление транзакцией (`START TRANSACTION`, `COMMIT`). Вставка записи в `Payments` и обновление статуса в `Bookings` выполняются как единое целое.

```
START TRANSACTION;  
  
INSERT INTO Payments (booking_id, amount, payment_date, method,  
status)  
VALUES (1, 5000.00, CURDATE(), 'Card', 'Completed');  
  
UPDATE Bookings  
SET status = 'Confirmed'  
WHERE id = 1 AND status = 'Pending';
```

```
COMMIT;
```

2.4. Типовые запросы и бизнес-логика

Запрос 1. Поиск свободных номеров на указанные даты.

Бизнес-процесс: Подбор номера для нового гостя.

```
SELECT r.id, r.room_number, rt.type_name, rt.base_price
FROM Rooms r
JOIN Room_Types rt ON r.room_type_id = rt.id
WHERE r.is_available = TRUE
AND r.id NOT IN (
    SELECT room_id FROM Bookings
    WHERE status IN ('Confirmed', 'Checked-In')
    AND check_in_date < '2024-12-20'
    AND check_out_date > '2024-12-15'
);
```

Запрос 2. Выгрузка информации о госте по номеру паспорта.

Бизнес-процесс: Идентификация клиента на стойке регистрации.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, email,
phone
FROM Guests
WHERE passport_number = '4510 123456';
```

Запрос 3. Отчет по выручке за текущий месяц.

Бизнес-процесс: Финансовый анализ для руководства отеля.

```
SELECT SUM(amount) AS total_revenue
FROM Payments
WHERE status = 'Completed'
AND MONTH(payment_date) = MONTH(CURDATE())
AND YEAR(payment_date) = YEAR(CURDATE());
```

Запрос 4. Список бронирований, у которых дата выезда — сегодня.

Бизнес-процесс: Формирование списка для службы хаускипинга (уборки).

```
SELECT b.id, CONCAT(g.first_name, ' ', g.last_name) AS guest,
r.room_number
FROM Bookings b
JOIN Guests g ON b.guest_id = g.id
JOIN Rooms r ON b.room_id = r.id
WHERE b.check_out_date = CURDATE() AND b.status = 'Checked-In';
```

Запрос 5. Подсчет загрузки отеля по типам номеров.

Бизнес-процесс: Аналитика для отдела маркетинга.

```

SELECT rt.type_name, COUNT(b.id) AS total_bookings
FROM Room_Types rt
LEFT JOIN Rooms r ON rt.id = r.room_type_id
LEFT JOIN Bookings b ON r.id = b.room_id AND b.status !=
'Cancelled'
GROUP BY rt.type_name;

```

2.5. Ролевая модель доступа

Для разграничения прав доступа созданы две учетные записи:

1. `hotel_admin` — полный доступ ко всем таблицам и объектам БД.
2. `hotel_receptionist` — доступ на чтение (SELECT) ко всем справочникам и представлениям, право на вставку (INSERT) и обновление (UPDATE) в таблицах Bookings и Payments, но запрет на удаление (DELETE) исторических данных.

```

CREATE USER 'hotel_admin'@'localhost' IDENTIFIED BY
'admin_secure_pass';
CREATE USER 'hotel_receptionist'@'localhost' IDENTIFIED BY
'recept_secure_pass';

GRANT ALL PRIVILEGES ON hotel_booking_db.* TO
'hotel_admin'@'localhost';

GRANT SELECT ON hotel_booking_db.* TO
'hotel_receptionist'@'localhost';
GRANT INSERT, UPDATE ON hotel_booking_db.Bookings TO
'hotel_receptionist'@'localhost';
GRANT INSERT, UPDATE ON hotel_booking_db.Payments TO
'hotel_receptionist'@'localhost';

FLUSH PRIVILEGES;

```

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была спроектирована и реализована база данных для системы бронирования отелей.

В теоретической главе был проведен анализ предметной области, обоснован выбор реляционной модели и рассмотрены принципы нормализации до 3НФ. Это позволило избежать избыточности данных и аномалий обновления, что критично для финансовых и исторических данных гостиничного бизнеса.

В практической главе были решены все поставленные задачи:

1. Разработана концептуальная модель и ER-диаграмма, включающая 5 связанных сущностей.
2. Спроектирована логическая и физическая структура БД в СУБД MySQL.
3. Реализованы все требуемые объекты БД:
 - Создано представление View_ActiveBookings для удобного вывода текущих заездов.
 - Написана пользовательская функция Func_CalculateBookingPrice для автоматического расчета стоимости.
 - Разработана хранимая процедура Proc_CheckInGuest, использующая локальные переменные, условие (IF) и обработчик исключений (SQLEXCEPTION).
 - Создан триггер trg_before_booking_insert, защищающий бизнес-логику от ввода некорректных дат.
 - Продемонстрирована транзакция обработки платежа с гарантией целостности данных.
4. Настроена ролевая модель (hotel_admin, hotel_receptionist) для разграничения прав доступа.
5. Сформировано 5 типовых SQL-запросов, покрывающих основные бизнес-процессы отеля.

СПИСОК ИСТОЧНИКОВ

1. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – М.: Вильямс, 2019. – 1328 с.
2. Кизема, Т. В. Проектирование баз данных / Т. В. Кизема. – М.: ФОРУМ : ИНФРА-М, 2021. – 240 с.
3. Шварц, Б. MySQL. Оптимизация производительности / Б. Шварц, П. Зайцев, В. Ткаченко. – СПб.: Питер, 2021. – 656 с.
4. Хомоненко, А. Д. Базы данных: учебник для вузов / А. Д. Хомоненко, В. М. Ципенко. – М.
5. Официальная документация MySQL 8.0 [Электронный ресурс]. – Режим доступа: <https://dev.mysql.com/doc/refman/8.0/en/> (дата обращения: 24.05.2026).

Приложение 1. Ссылка на репозиторий



<https://github.com/Kolpakch0k/kurs>